SIMATS ENGINEERING

Error Analysis Task

COURSE NAME: Software Testing **COURSE CODE: CSA3715**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 20%

Sl. No	Questions	Bloom's Level
1	A tester designs test cases for an online shopping system using equivalence partitioning but incorrectly groups input ranges (e.g., valid and invalid inputs mixed in one class). Additionally, boundary values are not considered. Analyze the given test design and identify all errors in equivalence class partitioning and boundary value analysis. Examine the root causes of these mistakes with proper reasoning. Propose corrected test cases with proper classification and boundary coverage. Justify how your corrections improve defect detection effectiveness.	3
2	A QA engineer performs random testing on a banking application and claims high coverage without using any structured techniques. Critical defects in transaction limits and boundary conditions are missed. Evaluate the testing approach and identify errors in strategy and coverage assumptions. Analyze the root causes behind missing critical defects. Suggest appropriate black box testing techniques to correct the approach. Justify how these techniques ensure better test adequacy and reliability.	3
3	A developer performs white box testing but creates an incomplete control flow graph, missing decision nodes and loops, resulting in poor path coverage. Analyze the errors in the control flow graph and path testing approach. Identify the root causes for inadequate test coverage. Propose corrections by constructing a proper control flow representation and identifying all independent paths. Justify how improved path testing ensures better coverage and defect detection.	5

4	A testing team reports 100% statement coverage for a module but fails to detect logical errors in conditional statements. Evaluate the adequacy of the testing performed and identify gaps in coverage criteria. Analyze why statement coverage alone was insufficient. Recommend improved white box testing strategies such as branch or path coverage. Justify how the revised approach enhances overall test effectiveness and error detection.	5
5	A tester uses Equivalence Class Partitioning for an age-validation field (0-100) but defines only two classes: Negative numbers and Positive numbers. Analyze the errors in this partitioning strategy. Identify why this fails to account for valid vs. invalid age limits and the absence of Boundary Value Analysis. Propose a corrected set of partitions and justify how they ensure more robust input validation.	3
6	A developer claims full Condition Coverage for a complex `if(A && B	
7	A QA team applies Decision Table Testing for a discount system but misses several combinations of "Premium Member" and "Coupon Applied," leading to price calculation errors. Analyze the errors in the construction of the decision table. Identify why the current approach is incomplete. Propose a revised, exhaustive decision table and justify its effectiveness in preventing production defects.	3
8	During unit testing of a looping structure, a tester checks only the first and last iterations (n=1 and n=max). Defects occurring at n=0 and n=max+1 are missed. Evaluate the errors in this Loop Testing strategy. Analyze the root causes behind these omissions. Recommend a corrected approach for testing simple and nested loops, and justify how it ensures path integrity.	5
9	An engineer utilizes Error Guessing as the primary testing technique for a safety-critical braking system, ignoring structured black box methods. Identify the errors in relying solely on ad-hoc techniques for high-risk modules. Analyze the impact on test adequacy. Suggest a combination of BVA and State Transition Testing to fix the strategy, justifying the increased safety assurance.	5

10	A tester designs a Control Flow Graph (CFG) for a module but fails to represent the "else" branches of multiple nested if statements, leading to 100% statement coverage but 0% branch coverage on error paths. Analyze the errors in graph construction. Identify the root causes for the missing nodes. Propose a corrected CFG and independent paths to ensure branch adequacy.	5
11	In an e-commerce checkout module, a tester applies State Transition Testing but fails to include "Invalid Transitions" (e.g., going from 'Payment Failed' to 'Shipped'). Analyze the gaps in the state transition table. Examine why negative state testing was ignored. Propose a corrected state diagram including all possible illegal transitions and justify how it prevents system state corruption.	3
12	A software module uses multiple input fields (Currency, Amount, Date). The tester uses Boundary Value Analysis on 'Amount' but treats 'Currency' as a single class, missing errors related to unsupported currency codes. Analyze the errors in identifying equivalence classes for non-numeric data. Propose a corrected test design incorporating Cause-Effect Graphing and justify its completeness.	3
13	A developer provides 100% Path Coverage for a module but forgets to initialize variables before they are used in certain paths, causing intermittent runtime crashes. Evaluate the limitation of path coverage in detecting data-flow anomalies. Analyze the root cause of these crashes. Recommend Data Flow Testing (definition-use chains) as a correction and justify its role in error prevention.	5
14	A testing team reports that a payroll system is fully tested because all "typical" employee profiles were processed successfully. However, the system crashes for "Contractors" with zero hours worked. Identify the errors in the Equivalence Partitioning of the employee roles. Analyze why "edge cases" were overlooked. Propose corrected test cases including Robustness Testing and justify the improved defect detection.	5

Code of Conduct:

*I **B.Hari Sai/192311098** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:B.Hari Sai

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

ANSWERS:

1. Equivalence Partitioning and Boundary Value Analysis in Online Shopping

Errors in Test Design: The tester has mixed valid and invalid inputs in the same equivalence class. An equivalence class should contain inputs expected to behave similarly. Boundary values have also been ignored, so defects at minimum and maximum limits may remain undetected.

Root Causes: The requirements were not divided into separate valid and invalid ranges, and the tester relied on representative values without checking the edges of each range.

Corrected Test Design: For an order amount allowed from ₹100 to ₹10,000, define classes as: below ₹100 (invalid), ₹100–₹10,000 (valid), and above ₹10,000 (invalid). Test representative values such as ₹50, ₹500, and ₹12,000. Apply boundary tests at ₹99, ₹100, ₹101, ₹9,999, ₹10,000, and ₹10,001. Add relevant checks for blank, non-numeric, and negative values.

Justification: Separate classes provide meaningful representative tests, while boundary tests target common off-by-one and comparison errors. Together they improve coverage and reduce the chance of invalid orders being accepted or valid orders being rejected.

2. Random Testing in a Banking Application

Errors in Strategy: Random testing alone does not guarantee requirement coverage. The claim of high coverage is unsupported because the tester has not mapped tests to transaction rules, limits, or risk areas.

Root Causes: Random values are unlikely to repeatedly hit exact transaction boundaries, combinations of account states, or rare error conditions. No measurable coverage criterion was used.

Corrected Approach: Use Equivalence Partitioning for valid and invalid account balances and transaction amounts; Boundary Value Analysis around minimum and maximum transaction limits; Decision Table Testing for combinations of account status, available balance, daily limit, and transaction type; and State Transition Testing for locked, active, and blocked accounts. Retain random testing as supplementary testing.

Justification: Structured techniques link tests to requirements and systematically cover boundaries and combinations. Coverage can be measured and reviewed, improving reliability and the likelihood of finding critical banking defects.

3. Incomplete Control Flow Graph and Path Testing

Errors in the CFG: The graph omits decision nodes and loop edges, so it does not represent every possible control transfer. Consequently, paths and branches are missing from the analysis.

Root Causes: The tester may have translated only the main sequence of statements and failed to inspect every condition, loop entry, loop exit, and error-handling route.

Corrected Representation and Paths: Construct the CFG with a node for each statement block and decision, directed edges for both true and false outcomes, and back-edges for loops. For a simple example: Start → input check; valid → process → output → End; invalid → error message → End. For a loop, include both the zero-iteration exit and the body-to-condition back-edge. Identify independent paths using a basis-path approach; the number of independent paths is measured by cyclomatic complexity, $V(G) = E - N + 2P$.

Justification: A complete CFG exposes missing branches and loop paths. Tests for each independent path increase structural coverage and help detect incorrect conditions, loop exits, and error-handling defects.

4. Limits of 100% Statement Coverage

Evaluation: 100% statement coverage means every executable statement ran at least once. It does not prove that every decision outcome or combination of conditions was tested.

Why It Is Insufficient: A test may execute the statement inside an if block but never test the false outcome. Logical errors such as using > instead of >=, or reversing a condition, can remain hidden.

Improved Strategy: Add Branch Coverage so every decision takes both true and false outcomes. Where a decision contains multiple Boolean conditions, use Condition Coverage or Modified Condition/Decision Coverage (MC/DC) when appropriate. Use path testing for critical logic, while recognizing that exhaustive path coverage may be infeasible when loops create many paths.

Justification: Branch and condition tests examine outcomes that statement coverage misses. They provide stronger evidence that decisions behave correctly and improve defect detection without claiming that any single metric proves the software is defect-free.

5. Correct Equivalence Classes for Age Validation (0–100)

Errors: The two classes, negative numbers and positive numbers, incorrectly group valid and invalid values. Positive numbers include ages above 100, while age 0 is a valid boundary if the specification permits it.

Corrected Partitions: Invalid class 1: age below 0. Valid class: age from 0 through 100 inclusive. Invalid class 2: age above 100. Also test non-integer values, text, blank input, and values that cannot be parsed if the field accepts only whole-number ages.

Boundary Tests: Test −1, 0, 1, 99, 100, and 101. Include malformed inputs such as "abc" and an empty value.

Justification: The corrected classes reflect the actual specification, and boundary tests target the points where validity changes. This helps detect incorrect comparisons, parsing errors, and acceptance of out-of-range ages.

6. Condition Coverage for if (A && B)

Evaluation: The question statement is incomplete after "if (A && B". Assuming the decision is A && B, full Condition Coverage requires each atomic condition A and B to evaluate to both true and false at least once.

Example Tests: Use (A=true, B=true) so the decision is true; (A=false, B=true) so A is false; and (A=true, B=false) so B is false. These tests exercise both outcomes of each condition. In short-circuit languages, B may not be evaluated when A is false, so choose tests that actually evaluate B as false.

Important Limitation: Condition Coverage alone does not guarantee Decision Coverage or prove that each condition independently changes the decision result. For safety-critical logic, consider MC/DC, where each condition is shown to independently affect the overall decision.

Justification: Explicitly testing both outcomes of each condition reduces the chance of missed Boolean logic defects. The test objective should be stated clearly and verified against actual evaluation behavior.

7. Decision Table Testing for a Discount System

Errors in the Existing Table: The tester has omitted combinations of Premium Member and Coupon Applied. A partial table can miss interactions, leading to incorrect discounts, double discounts, or discounts being denied incorrectly.

Corrected Exhaustive Decision Table: Assume the rule is: premium members receive 10%; a valid coupon adds 5%; both benefits may be combined. The four rules are: (1) Premium=No, Coupon=No → 0%; (2) Premium=No, Coupon=Yes → 5%; (3) Premium=Yes, Coupon=No → 10%; (4) Premium=Yes, Coupon=Yes → 15%. If business rules prohibit stacking or coupons have eligibility conditions, encode those rules explicitly instead.

Test Cases: Test all four combinations using the same eligible cart amount. Also test an invalid or expired coupon, a cart below the coupon minimum, and rounding or maximum-discount rules if specified.

Justification: A complete decision table ensures every combination is considered. It makes missing rules visible to stakeholders and provides traceable tests for preventing price calculation defects.

8. Loop Testing: Zero, Boundary, and Nested Iterations

Errors and Root Causes: Testing only n=1 and n=max ignores zero iterations and one-above-maximum behavior. This misses skipped-loop errors, incorrect loop entry conditions, off-by-one errors, and failure to reject excessive iteration counts.

Corrected Simple-Loop Tests: For a permitted count from 1 to max, test n=0, n=1, n=2, n=max−1, n=max, and n=max+1. Confirm whether zero is valid from the specification; if it is valid, verify correct no-op behavior rather than rejection. Also test invalid or non-numeric counts where applicable.

Nested and Concatenated Loops: For nested loops, start with the smallest practical counts, then vary the inner loop while holding the outer loop small. Test boundary values for each loop independently, then selected combinations such as minimum/minimum, maximum/minimum, minimum/maximum, and realistic maximum/maximum. For dependent loops, test their interaction and termination conditions.

Justification: These tests cover loop entry, exit, repetition, and limit enforcement. They reveal off-by-one and termination defects while avoiding the impractical cost of testing every possible iteration combination.

9. Error Guessing in a Safety-Critical Braking System

Errors in Relying Only on Error Guessing: Error Guessing depends on tester experience and is not systematic. Used alone, it may overlook required behavior, rare states, sensor limits, timing constraints, and combinations that affect braking safety.

Impact: Unstructured testing gives weak evidence of completeness and can leave dangerous failure modes untested. A successful set of ad-hoc tests does not establish safety or compliance.

Corrected Strategy: Use Boundary Value Analysis for sensor thresholds, braking pressure, speed, response time, and permitted operating ranges. Use State Transition Testing for states such as normal braking, ABS active, sensor fault, emergency braking, and system recovery. Test valid and invalid transitions, including sensor failures and transitions under changing speed or grip conditions. Add requirements-based tests, fault injection, and appropriate safety analysis.

Justification: BVA systematically checks threshold behavior, while State Transition Testing verifies behavior across operating modes and failures. Combining them with risk-based safety testing provides stronger evidence than error guessing alone.

10. CFG Missing Else Branches in Nested Decisions

Errors in Graph Construction: Omitting else branches removes false-outcome edges from the CFG. The graph therefore fails to represent error paths and may report misleading coverage results. Statement coverage can still reach 100% if statements are executed through other routes, while branches remain untested.

Root Causes: The tester may have treated if statements as single actions, overlooked implicit fall-through behavior, or failed to map error handling and nested conditions separately.

Corrected CFG and Independent Paths: For a nested example, use: Start → test A. If A=true, test B; if B=true, perform action X; if B=false, handle error B. If A=false, handle error A. Join the paths at End. Test at least these independent routes: A=true/B=true; A=true/B=false; and A=false. For more complex logic, include every true and false edge and calculate cyclomatic complexity using $V(G)=E-N+2P$ to guide the basis paths.

Justification: Representing both outcomes makes error paths visible and enables branch coverage. Tests on each independent route can reveal missing error messages, incorrect recovery, and unsafe fall-through behavior.

11. State Transition Testing for E-commerce Checkout

Gaps in the State Model: The tester has not modeled invalid transitions, such as moving directly from Payment Failed to Shipped. Testing only valid flows cannot show whether the system safely rejects illegal state changes.

Corrected State Model: A typical valid flow is Cart → Checkout → Payment Pending → Payment Successful → Shipped → Delivered. Payment Pending may transition to Payment Failed or Payment Successful. Payment Failed may transition to Payment Pending after a retry or return to Cart, according to requirements. It must not transition directly to Shipped. Cancelled orders must not be shipped; Delivered orders must not return to Payment Pending.

Negative Tests: Attempt illegal transitions such as Payment Failed → Shipped, Cart → Delivered, Cancelled → Shipped, and Delivered → Payment Pending. Verify that each is rejected, logged where required, and leaves the order state unchanged. Include retries, duplicate payment callbacks, and repeated shipping requests.

Justification: A complete transition table tests both permitted and forbidden changes. Negative transition tests help prevent inconsistent order records, duplicate fulfillment, and state corruption.

12. Currency, Amount, and Date: Cause-Effect Graphing

Errors in Test Design: Applying BVA only to Amount ignores categorical and format-related input risks. Currency is not one uniform class: supported codes, unsupported codes, incorrect formats, and missing values can produce different behavior. Date inputs also have valid, invalid, and boundary cases.

Corrected Equivalence Classes: Currency: each supported currency or equivalent behavior group; unsupported code; malformed code; blank value. Amount: negative, zero, valid range, above maximum, excessive decimal places, non-numeric value. Date: valid date, impossible date, malformed date, past/future date where restricted, and minimum/maximum allowed dates.

Cause-Effect Design: Define causes such as currency supported, amount valid, date valid, and currency-amount combination permitted. Define effects such as accept transaction, reject currency, reject amount, or reject date. Build a cause-effect graph and convert it into a decision table so interactions are tested, including supported currency with valid amount, unsupported currency with valid amount, valid currency with invalid amount, and invalid date with otherwise valid inputs.

Justification: Equivalence classes cover distinct input behaviors, BVA targets numeric and date limits, and Cause-Effect Graphing exposes combinations that affect the outcome. Together they provide more complete validation than testing Amount alone.

13. Path Coverage and Data-Flow Anomalies

Evaluation: 100% Path Coverage, even if accurately measured, does not guarantee that variables are initialized correctly before use. Path coverage focuses on control flow, not necessarily on the validity of every variable definition and use.

Root Causes: A variable may be used before assignment on a particular route, overwritten before use, or assigned only on some branches. These are data-flow problems that may produce intermittent crashes or unpredictable results.

Corrective Data-Flow Testing: Identify each variable's definition (assignment) and use (computation or condition). Trace definition-use (DU) chains along feasible paths. Test paths where a variable is defined and then used, where a branch skips its definition, and where a value is redefined before use. Add static analysis and compiler warnings to detect uninitialized-variable risks.

Justification: DU testing directly examines whether values are defined before use and whether assignments reach the intended uses. Combined with control-flow tests, it can expose runtime defects that path coverage alone may miss.

14. Payroll System: Contractors with Zero Hours

Errors in Equivalence Partitioning: Testing only typical employee profiles leaves out the Contractor role and the special case of zero hours. Employee type and hours worked are separate inputs and should be partitioned according to the payroll rules.

Root Causes: The test design overemphasized common cases and did not consider unusual but valid profiles, invalid combinations, or minimum values. The software may assume that every employee has positive hours or may apply a salary-only calculation to contractors.

Corrected Test Cases: Test permanent employee with normal hours; contractor with positive hours; contractor with zero hours; contractor with minimum billable hours; employee with zero hours where permitted; negative hours; hours above the permitted maximum; missing employee type; and unknown employee type. Verify whether zero hours should produce zero pay, a minimum payment, or a validation error according to the specification. Include Robustness Testing for malformed records and boundary values.

Justification: Separating employee roles and hours ranges ensures each behavior class is represented. Robustness and boundary tests expose assumptions that fail for unusual inputs, reducing payroll crashes and incorrect payments.